

CSCI 332, Fall 2026

Homework 2

Due Tuesday, September 8 Anywhere on Earth (6am Wednesday)

Submission Requirements

- Type or clearly hand-write your solutions into a PDF format so that they are legible and professional. Submit your PDF on Gradescope.
- Do not submit your first draft. Type or clearly re-write your solutions for your final submission. If your submission is not legible, we will ask you to resubmit.
- Use Gradescope to assign problems to the correct page(s) in your solution. If you do not do this correctly, we will ask you to resubmit.

Academic Integrity

Remember, you may access **any** resource in preparing your solution to the homework. However, you **must**

- write your solutions in your own words, and
- credit every resource you use (for example: “Bob Smith helped me on this problem. He took this course at UM in Fall 2020”; “I found a solution to a problem similar to this one in the lecture notes for a different course, found at this link: www.profzeno.com/agreatclass/lecture10”; “Here is a link to the full transcript of my chat with ChatGPT about the problems on this HW”.) Ask if you need any clarifications about how to properly cite your sources!

1. (0 points) List the resources you used for each problem in this homework. If you used no resources except lecture notes and videos and the linked textbook, you can say “none”. If you used an AI tool, you should put a link to your full conversation here, and you must use the [provided prompt](#) to guide the AI to act as a tutor, not an answer-provider.

You will be asked to resubmit your homework if this section is blank.

Remember that no matter what resources you consulted, you must write your solutions *in your own words*.

2. (1 points) In class, we said that the exact running time of an algorithm is the number of primitive operations it takes on an input of size n . (Usually, we disregard this precise running time and instead give some simpler function such that the precise function is big O if the other function.) The course textbook (Algorithms by Jeff Erickson) suggests defining the precise running time in a different way. What is that different way? *Hint: Look at Section, 0.6 Analyzing Algorithms.*
3. Consider the following basic problem. You’re given an array A consisting of n integers $A[1], A[2], \dots, A[n]$. You’d like to output a two-dimensional n -by- n array B in which $B[i, j]$ (for $i < j$) contains the sum of array entries $A[i]$ through $A[j]$ —that is, the sum $A[i] + A[i + 1] + \dots + A[j]$. (The value of array entry $B[i, j]$ is left unspecified whenever $i \geq j$, so it doesn’t matter what is output by those values.)
 - (a) (1 point) Give the resulting B for $A = [4, -9, 12, 2]$.
 - (b) (3 point) Here is an algorithm for this problem:

```
For  $i = 1, 2, \dots, n$ 
  For  $j = i + 1, i + 2, \dots, n$ 
    Add up array entries  $A[i]$  through  $A[j]$ 
    Store the result in  $B[i, j]$ 
  Endfor
Endfor
```

Give a function f that counts the number of primitive operations performed by the algorithm on an input of size n . This should be an exact count, not an asymptotic one! If you aren’t sure which operations count as primitive operations, either declare which ones you are considering to be primitive operations, or ask.

- (c) (2 points) Choose a function g that has the following properties:
 - The $f(n)$ you gave above is $O(g(n))$
 - The $f(n)$ you gave above is $\Omega(g(n))$
 - (d) (2 points) Although the algorithm you analyzed in parts (b)–(c) is the most natural way to solve the problem—after all, it just iterates through the relevant entries of the array B , filling a value for each—it contains some highly unnecessary sources of inefficiency. Give a different algorithm to solve this problem, with an asymptotically better running time.
4. (1 point) What does “asymptotically better” mean?